



Higher Diploma in Computer Science

Computer Systems & Networks



What are RegEx?

Used by

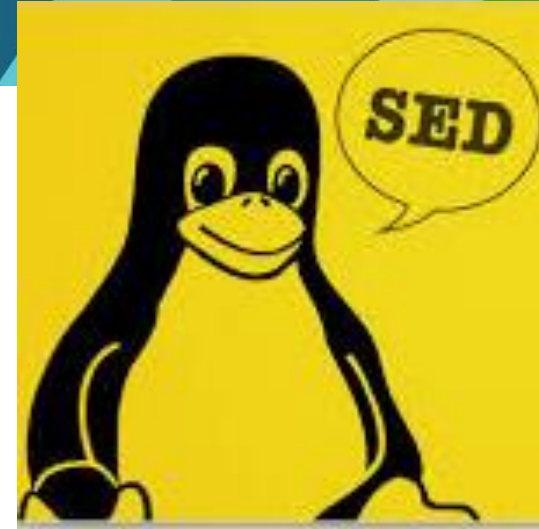
- grep
- tr
- sed etc.

RECALL: this slide from notes on Regular Expressions which mentions sed

What is `sed`?

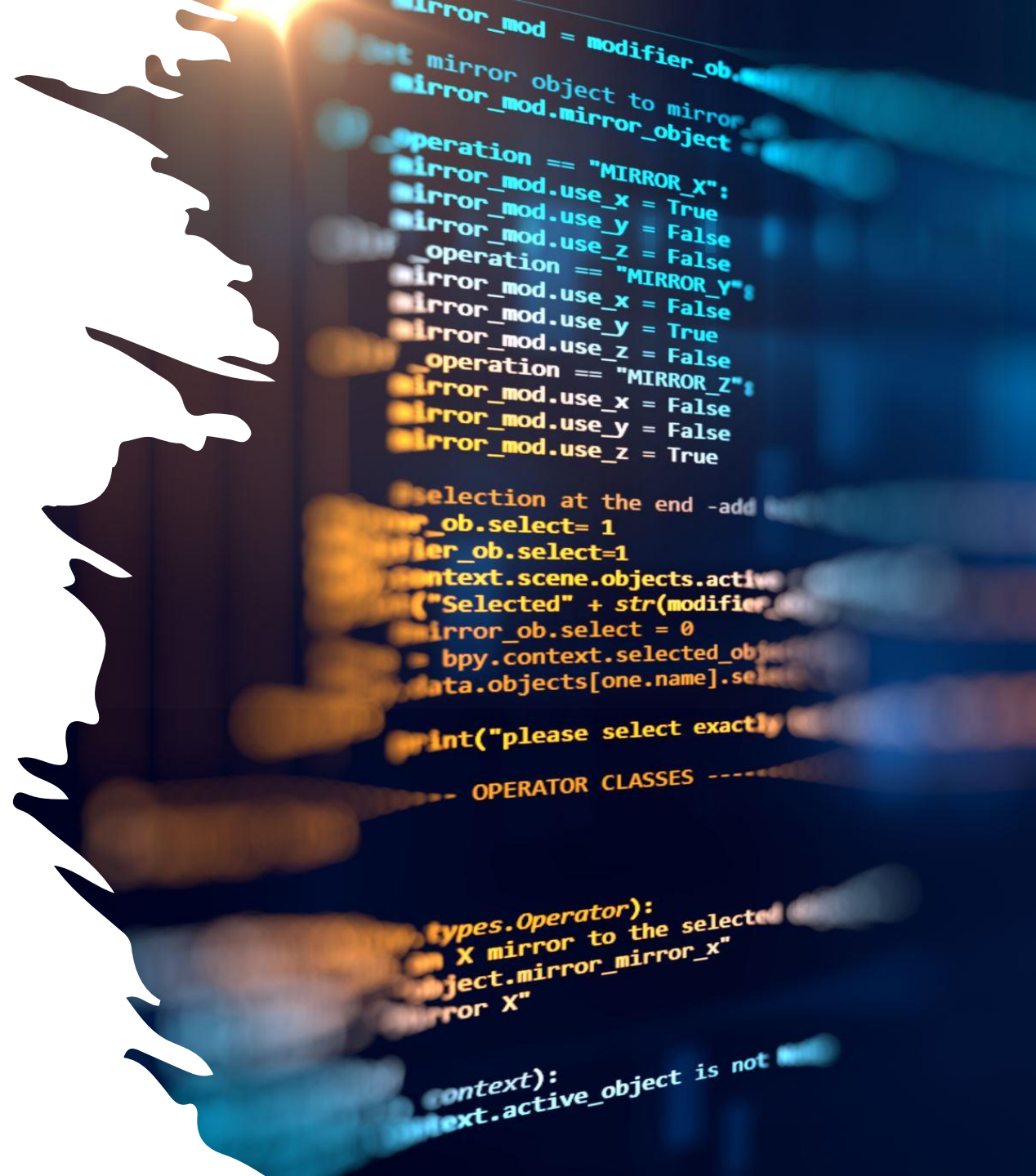
sed

Stream editor



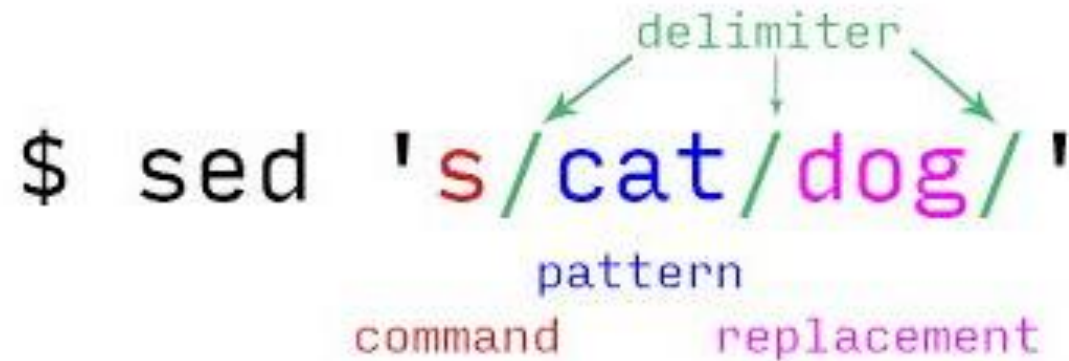
\$sed stream editor

- **sed** is a special editor for **modifying files** automatically
 - Can make global changes to a large file
 - Allows a script make changes to a file
 - Can filter text in a pipeline
- **sed** requires **an input file** and a **command** that lets **sed** know what actions to apply to the file



```
mirror_mod = modifier_ob.  
Set mirror object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly  
-- OPERATOR CLASSES --  
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
context):  
context.active_object is not
```

syntax



The diagram illustrates the syntax of the `sed` command. It shows the command `$ sed 's/cat/dog/'` with color-coded components and arrows pointing to their labels:

- `s` is labeled **command** (red text).
- `/` is labeled **delimiter** (green text).
- `cat` is labeled **pattern** (blue text).
- `/` is labeled **delimiter** (green text).
- `dog` is labeled **replacement** (purple text).
- `/` is labeled **delimiter** (green text).

- Commands begin with a letter i.e. `s` for substitute
- Using `sed`'s substitute operator, substitute the word “cat” with “dog”

sed common operators

\$ sed 's/cat/dog/'

The diagram shows the command `s/cat/dog/` with labels: `s` is the **command**, `cat` is the **pattern**, `dog` is the **replacement**, and the slashes `/` are the **delimiter**.

Operator	Name	Effect
[address-range]/p	print	Print [specified address range]
[address-range]/d	delete	Delete [specified address range]
s/pattern1/pattern2/	substitute	Substitute pattern2 for first instance of pattern1 in a line

Conventions: *delimiter* and *global* changes

The slash character is often used as delimiter

sed will accept any character that immediately follows the command as the delimiter:

```
$echo "cat" | sed 's_cat_dog_'
```

Find & replace

\$sed -e 's/cat/dog/' my_file

\$ sed 's/cat/dog/'

The diagram illustrates the components of the `s/cat/dog/` command within single quotes. The `s` is labeled as the **command**. The `/` is labeled as the **delimiter**. The `cat` is labeled as the **pattern**. The `dog` is labeled as the **replacement**.

echo every line from *my_file* to STDOUT, changing ***the first occurrence*** of the string 'cat' on each line into the string 'dog'.

-e causes the next string to be interpreted as an editing instruction
optional if passing one instruction

Global changes

sed is line-oriented, so if you wish to change ***every occurrence*** on each line, then you need to make it a 'greedy' global search & replace like so:

```
$sed -e 's/input/output/g' my_file
```

A large, bold, black lowercase letter 'g' is centered within a light gray square. The 'g' is a simple, sans-serif font.

Global changes

```
$ echo $PATH
```

The diagram illustrates the components of the `sed` command `$ sed 's/cat/dog/'`. The `s` is labeled as the **command**. The `/` following `s` is the **delimiter**. The text `cat` is the **pattern**. The text `dog` is the **replacement**. The final `/` is also a **delimiter**.

You want a directory listing, where you replace each colon with a space..

```
$ echo $PATH | sed 's/:/ /g'
```

A large, bold, black lowercase letter 'g' is displayed on a light gray square background.

sed addresses

\$ sed 's/cat/dog/'

The diagram shows the command 's/cat/dog/' with labels: 's' is the command, '/' is the delimiter, 'cat' is the pattern, and 'dog' is the replacement.

- Most commands in *sed* may be preceded by an address
 - specifies which line(s) of the input stream will be edited

\$echo "cat" | sed '1s/cat/dog/'

\$dog

#replace on 1st line only

\$echo "cat" | sed '2s/cat/dog/'

#No editing, no line 2

- A *sed* command can specify zero, one, or two addresses

sed addresses

- An address can be a line number, a line addressing symbol, or a regular expression that describes a pattern.
 1. ***no address*** specified → command is applied to each line.
 2. only ***one*** address → command is applied to any line matching the address.
 3. ***comma-separated addresses*** → An address range where the command is performed on the first matching line and all succeeding lines up to and including a line matching the second address.
 4. address ***followed by !*** → command is applied to all lines that do not match the address.

sed addresses

Address	Description
<i>n</i>	A line number where <i>n</i> is a positive integer.
<i>\$</i>	The last line.
<i>/regex/</i>	Lines matching a POSIX basic regular expression. Note that the regular expression is delimited by slash characters. Optionally, the regular expression may be delimited by an alternate character, by specifying the expression with <i>\cregexpc</i> , where <i>c</i> is the alternate character.
<i>addr1,addr2</i>	A range of lines from <i>addr1</i> to <i>addr2</i> , inclusive. Addresses may be any of the single address forms above.
<i>first~step</i>	Match the line represented by the number <i>first</i> , then each subsequent line at <i>step</i> intervals. For example 1~2 refers to each odd numbered line, 5~5 refers to the fifth line and every fifth line thereafter.
<i>addr1,+n</i>	Match <i>addr1</i> and the following <i>n</i> lines.
<i>addr!</i>	Match all lines except <i>addr</i> , which may be any of the forms above.

Printing in *sed*

```
Caroline@ICTSkills>~$ sed -n '7,9p' distros.txt
Ubuntu 8.04 24/04/2008
Fedora 8 08/11/2007
SUSE 10.3 04/10/2007

Caroline@ICTSkills>~$ sed -n '/SUSE/p' distros.txt
SUSE 10.2 07/12/2006
SUSE 10.2 07/12/2006
SUSE 11.0 19/06/2008
SUSE 11.0 19/06/2008
SUSE 10.3 04/10/2007
SUSE 10.3 04/10/2007
SUSE 10.1 11/05/2006
```

```
Caroline@ICTSkills>~$ sed -n '/SUSE/!p' distros.txt
```

- You can also use a **pattern-range** form, where the first regexp defines the start of the range, and the second the stop:

```
$sed -n -e '/caroline/,/rosanne/p' employee.txt
```


sed -n (no print)

The "-n" option will not print anything unless an explicit request to print is found. I mentioned the "/p" flag to the substitute command as one way to turn printing back on. Let me clarify this. The command

```
sed 's/PATTERN/&/p' file
```

acts like the *cat* program if PATTERN is not in the file: e.g. nothing is changed. If PATTERN is in the file, then each line that has this is printed twice. Add the "-n" option and the example acts like *grep*:

```
sed -n 's/PATTERN/&/p' file
```

Nothing is printed, except those lines with PATTERN included.

The long argument of the -n command is either

```
sed --quiet 's/PATTERN/&/p' file
```

or

```
sed --silent 's/PATTERN/&/p' file
```

- you must use **p** command of print flag to explicitly specify what is to be printed

sed operators

Notation	Effect
8d	Delete 8th line of input.
/^\$/d	Delete all blank lines.
1,/^\$/d	Delete from beginning of input up to, and including first blank line.
/Jones/p	Print only lines containing "Jones" (with -n option).
s/Windows/Linux/	Substitute "Linux" for first instance of "Windows" found in each input line.
s/BSOD/stability/g	Substitute "stability" for every instance of "BSOD" found in each input line.
s/ *\$//	Delete all spaces at the end of every line.
s/00*/0/g	Compress all consecutive sequences of zeroes into a single zero.
echo "Working on it." sed -e '1i How far are you along?'	Prints "How far are you along?" as first line, "Working on it" as second.
5i 'Linux is great.' file.txt	Inserts 'Linux is great.' at line 5 of the file file.txt.
/GUI/d	Delete all lines containing "GUI".
s/GUI//g	Delete all instances of "GUI", leaving the remainder of each line intact.

Removing blank lines in a file

\$ sed 's/cat/dog/'

command pattern replacement

`$sed -e /^$/d "$1"` the same as

'^' indicates the beginning of line

`$sed -e '/^$/d' filename`

'\$' the end.

`^$` matches lines with nothing between

`"$1"` Adding **quote marks** around the argument permits whitespace and special characters in the filename.

Note that this script doesn't actually change the target file.

- If you need to do that, redirect its output (>)

Redirecting output

\$ sed 's/cat/dog/'

The diagram illustrates the components of the `sed` command `'s/cat/dog/'`. The `s` is labeled as the **command**. The `/` following `s` is the **delimiter**. The text `cat` is the **pattern**. The text `dog` is the **replacement**. The final `/` is also the **delimiter**.

- By default the output is written to stdout.
- May redirect this to a new file,

```
sed -e 's/input/output/' my_file > new_file
```

- or if you want to edit the existing file in place you should use the `-i` flag

```
sed -i -e 's/input/output/' my_file
```

Escape Symbol \

- You want a shell scripts to refer to **/usr/local/bin** and not **/bin** any more:

```
sed -e 's/\bin/\usr\local\bin/' my_script > new_script
```

\$ sed 's/cat/dog/'

The diagram shows the command 's/cat/dog/' with labels: 's' is the command, '/' is the delimiter, 'cat' is the pattern, and 'dog' is the replacement.

Wildcards

- dot (.) matches any **single** character in regular expressions
- * match any **string** (any single character zero or more times)
- ? match "zero or one of the previous thing", and
- + match "one or more ... "

Wildcards

- You want to search for every line that *starts* with a *number* in your file
- Now, you want to add brackets around the number (replace that number by brackets)

Complete this with your ***employee.txt*** AND redirect your output to another file, use this as a starting point:

```
sed -e 's/[0-9]*/&/' my_file
```

RECAP: syntax

The general syntax is

```
$sed -e '/pattern/ command' my_file
```

Where '***pattern***' is a regexp and

command' can be one of many i.e.

's' = search & replace, or

'p' = print, or

'd' = delete, or

'i'=insert, or

'a'=append, etc

grep/sed – differences?

- **grep**. A fast and powerful pattern search tool that can be easily combined with other filters to find results and customise the display, even if the main aim is to search for matches.
- **sed**. A fast stream editor, able to search for a pattern and apply the given transformations and/or commands; main aim: modifying the text in the stream.

Many tutorials e.g.:

